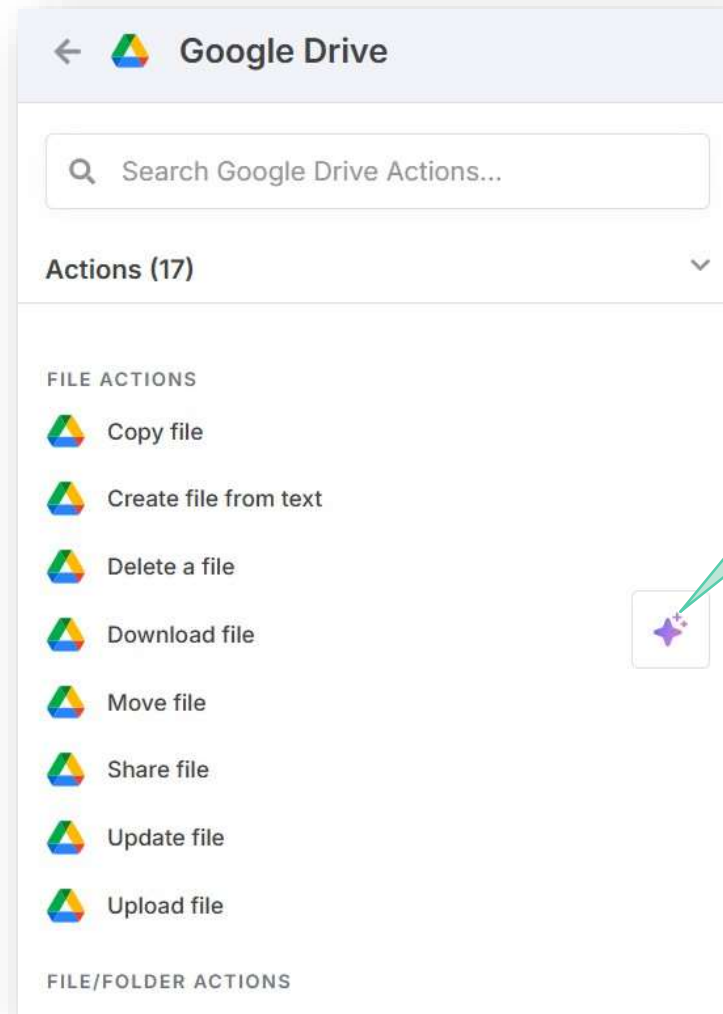


RAG in n8n

Advanced AI Agents with n8n

RAG Workflow



Google drive, download file

RAG Workflow

The screenshot displays a workflow editor interface with three main panels: INPUT, Google Drive (Parameters), and OUTPUT.

INPUT Panel: Shows a trigger event "When clicking 'Test workflow'" with a status of "1 item". Below the trigger, it indicates "No fields - item(s) exist, but they're empty" and provides a link to "Variables and context".

Google Drive Panel (Parameters): This panel is used to configure the Google Drive node. It includes the following settings:

- Credential to connect with:** Google Drive account
- Resource:** File
- Operation:** Download
- File:** By URL (selected) with the URL <https://drive.google.com/file/d/1AGy5ijt>
- Options:** No properties (Add option button)

OUTPUT Panel: Shows the result of the download operation. It displays a "data" object with the following properties:

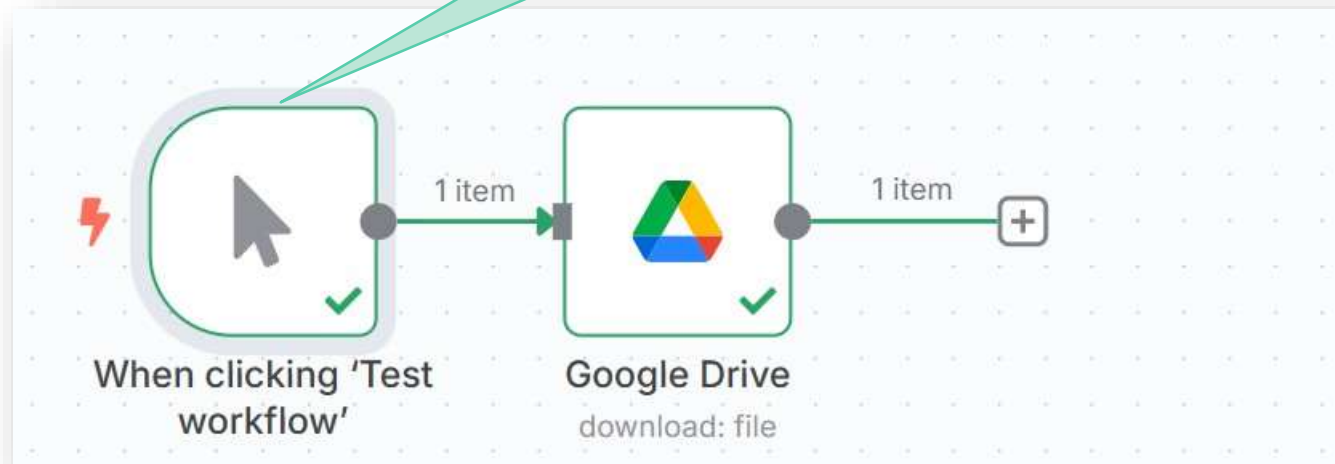
- File Name: COI.pdf
- File Extension: pdf
- Mime Type: application/pdf
- File Size: 3.5 MB

A "Download" button is visible below the data object.

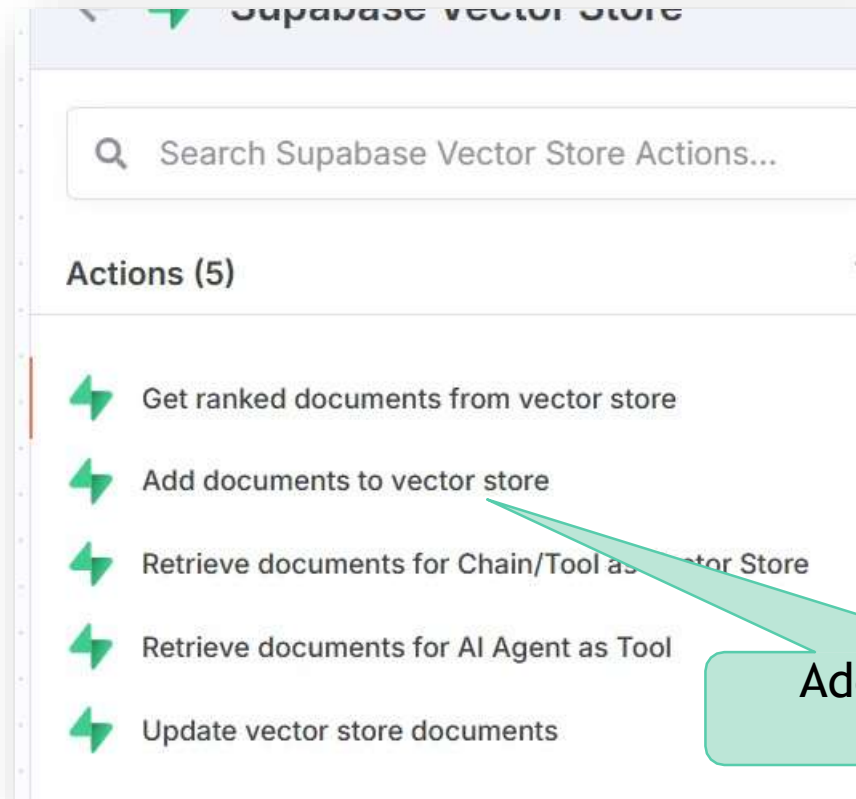
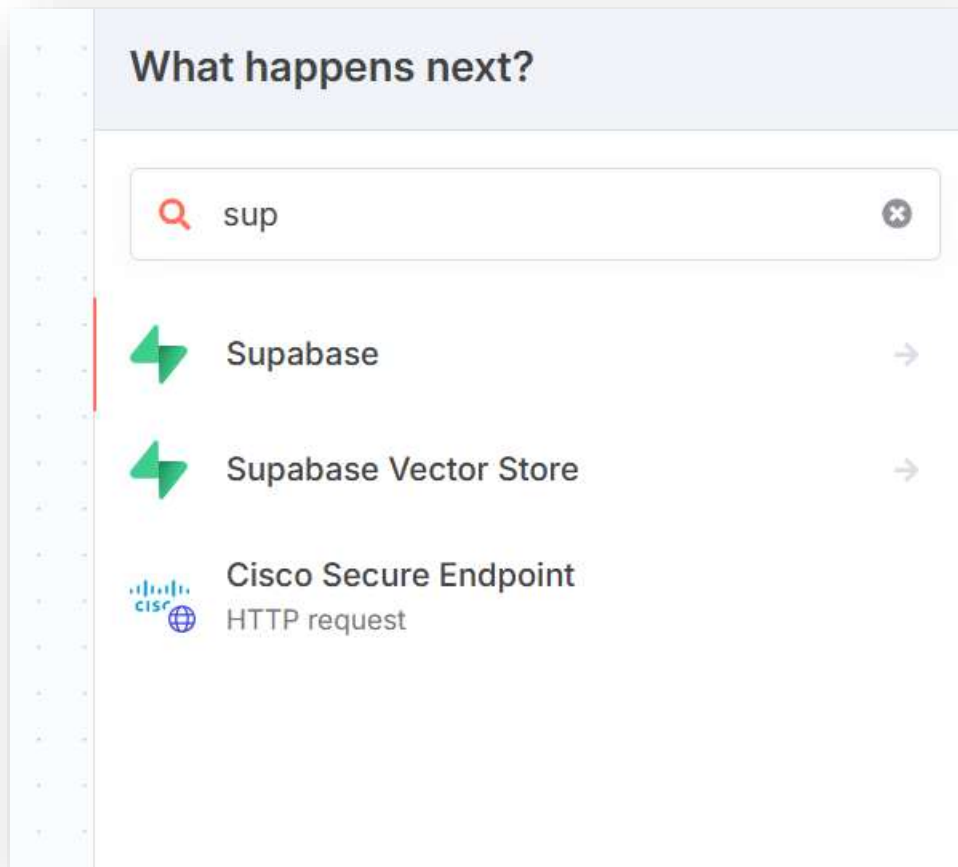
A green callout box with the text "By URL or File id" points to the "By URL" dropdown and the URL field in the Google Drive Parameters panel.

RAG Workflow

This trigger node will be automatically created.



RAG Workflow



Add documents to the vector store

RAG Workflow

 Supabase account 2
Supabase API

Save

Connection

Need help filling out these fields? [Open docs](#)

Sharing

Details

 Ask Assistant for setup instructions

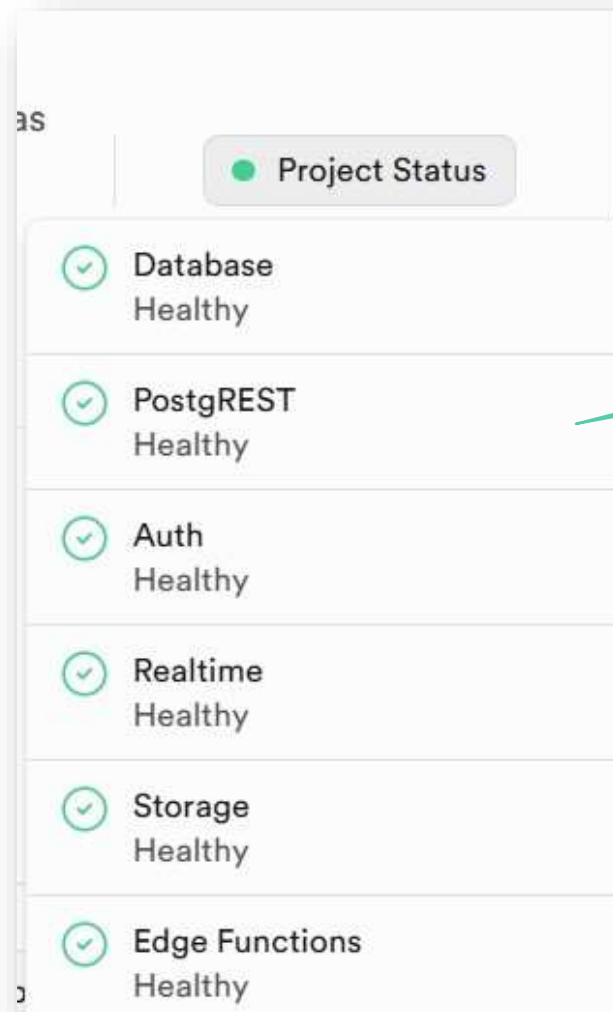
Host

Service Role Secret

 Enterprise plan users can pull in credentials from external vaults. [More info](#)

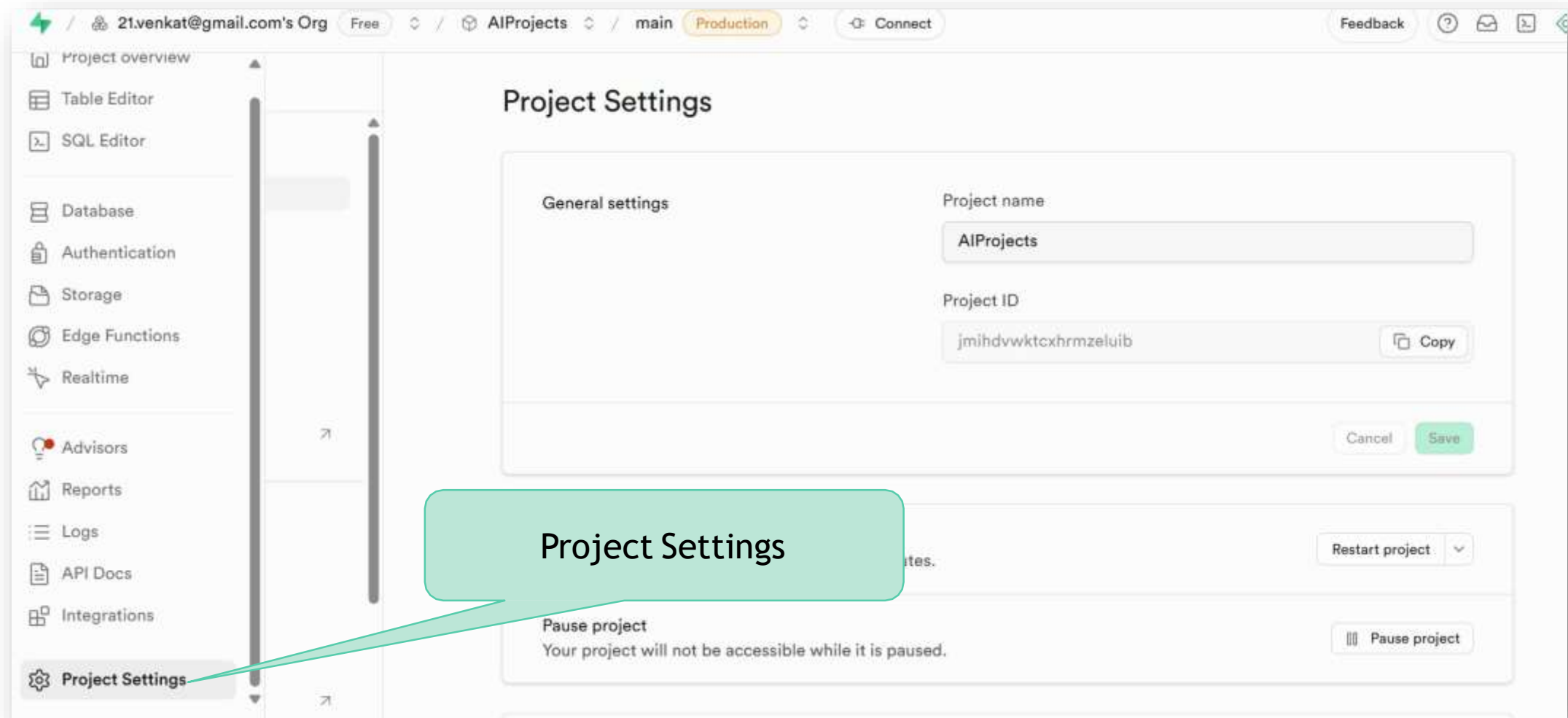
Get host and service role secret from Supabase

RAG Workflow



Check the project status -
it must be green

RAG Workflow



RAG Workflow

The screenshot displays the Supabase dashboard interface. The top navigation bar includes the user email '21.venkat@gmail.com's Org', a 'Free' tier indicator, the project name 'AIProjects', the environment 'main', and a 'Production' status tag. A 'Connect' button is also present. The left sidebar contains a 'Settings' menu with options like 'API Keys', 'JWT Keys', 'Add Ons', and 'Vault'. Below this is a 'CONFIGURATION' section with 'Database', 'Data API' (highlighted), 'Authentication', 'Storage', 'Edge Functions', and 'Log Drains'. At the bottom is a 'BILLING' section with 'Subscription' and 'Usage'. The main content area is titled 'API Settings' and features a 'Project URL' section with a text input containing 'https://jmihdvwktcxhrmzeluib.supabase.co' and a 'Copy' button. A callout bubble points to this URL with the text 'This project url is the host'. Another callout bubble points to the 'Data API' menu item with the text 'Project Settings>>Data API'. The bottom of the 'API Settings' section shows 'Data API Settings' with 'Docs' and 'Harden Data API' buttons.

Settings

- API Keys **NEW**
- JWT Keys **NEW**
- Add Ons
- Vault **Alpha**

CONFIGURATION

- Database
- Data API**
- Authentication
- Storage
- Edge Functions
- Log Drains

BILLING

- Subscription
- Usage

API Settings

Project URL

Source Primary Database

URL

<https://jmihdvwktcxhrmzeluib.supabase.co> **Copy**

RESTful endpoint for querying and managing your database

API keys and JWT settings have moved to their respective pages.

Compute and Disk Infrastructure Integrations **API Keys NEW** JWT Keys **NEW** Add Ons

Data API Settings **Docs** **Harden Data API**

This project url is the host

Project Settings>>Data API

RAG Workflow

 Supabase account 2
Supabase API

Save

×

Connection

Need help filling out these fields? [Open docs](#)

Sharing

Details

Ask Assistant for setup instructions

Host

https://your_account.supabase.co

Service Role Secret

.....

 Enterprise plan users can pull in credentials from external vaults. [More info](#)

project url from supabase
is the host

RAG Workflow

The screenshot shows the Google Cloud Project Settings page for the project 'AIProjects'. The left sidebar contains a 'Settings' menu with options like 'API Keys', 'JWT Keys', 'Add Ons', and 'Vault'. The main content area is titled 'API Keys' and includes a description: 'Configure API keys to securely control access to your project'. There are two tabs: 'Legacy API Keys' and 'API Keys'. The 'API Keys' tab is active, showing a list of API keys. The first key is labeled 'anon' and 'public', with a value 'eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ...' and a note: 'This key is safe to use in a browser if you have enabled row level security on your tables and configured policies. Prefer using Publishable API keys instead.' The second key is labeled 'service_role' and 'secret', with a value '**** *' and a note: 'This key has the ability to bypass Row Level Security. Never share it publicly. If leaked, generate a new JWT secret immediately. Prefer using Secret API keys instead.' A green callout bubble points to the 'API Keys' menu item in the sidebar, and another points to the 'Reveal' button for the 'service_role' key.

Project settings>>API Keys

Reveal for service role secret

RAG Workflow

Need to create a vector database table for us to store the vector

Click on Docs to copy and paste the code

The screenshot displays a workflow interface with two main components. On the left, a 'data' block shows file details for 'COI.pdf' (3.92 MB) with a 'Download' button. On the right, a 'Supabase Vector Store' configuration panel is active, featuring tabs for 'Parameters', 'Settings', and 'Docs'. The 'Parameters' tab is selected, showing fields for 'Credential to connect with' (Supabase account), 'Operation Mode' (Insert Documents), 'Table Name' (From list / Choose...), and 'Embedding Batch Size' (200). A 'Test step' button is visible in the top right of the configuration panel. A 'Docs' link is also present. The background shows a blurred 'OUTPUT' section.

RAG Workflow

Integrations > Built-in nodes > Cluster nodes > Root nodes

Supabase Vector Store node

Use the Supabase Vector Store to interact with your Supabase database as [vector store](#). You can insert documents into a vector database, get documents from a vector database, retrieve documents to provide them to a retriever connected to a [chain](#), or connect it directly to an [agent](#) to use as a [tool](#).

On this page, you'll find the node parameters for the Supabase node, and links to more resources.

Credentials

You can find authentication information for this node [here](#).

Parameter resolution in sub-nodes

Sub-nodes behave differently to other nodes when processing multiple items using an expression.

Most nodes, including root nodes, take any number of items as input, process these items, and output the results. You can use expressions to refer to input items, and the node resolves the expression for each item in turn. For example, given an input of five `name` values, the expression `{{ $json.name }}` resolves to each name in turn.

In sub-nodes, the expression always resolves to the first item. For example, given an input of five `name` values, the expression `{{ $json.name }}` always resolves to the first name.

Supabase provides a [quickstart for setting up your vector store](#). If you use settings other than the defaults in the quickstart, this may affect parameter settings in n8n. Make sure you understand what you're doing.

Click here to copy the code to setup the vector store

RAG Workflow v

Copy these SQL Commands

AI & Vectors > Third-Party Tools > LangChain

LangChain

LangChain is a popular framework for working with AI, Vectors, and embeddings. LangChain supports using Supabase as a vector store, using the `pgvector` extension.

Initializing your database

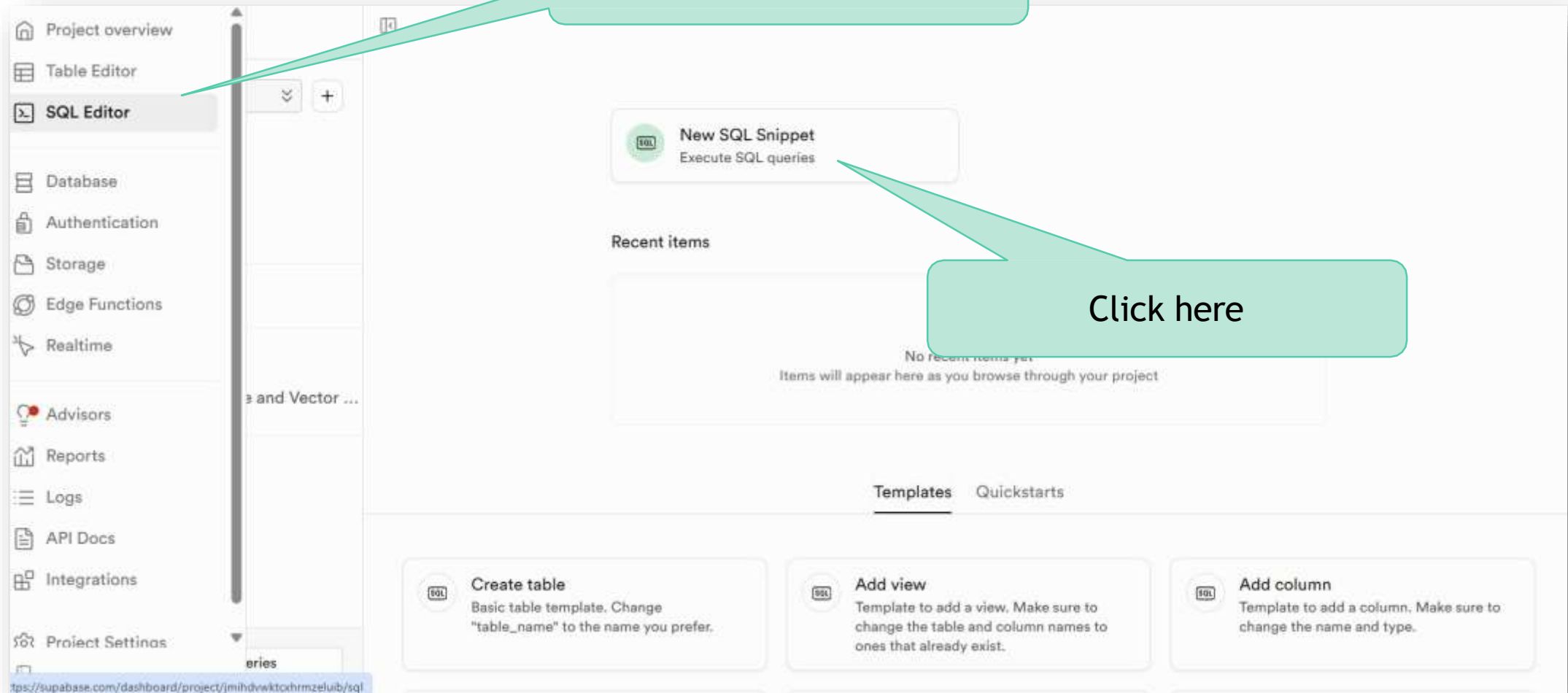
Prepare your database with the relevant tables:

Dashboard SQL

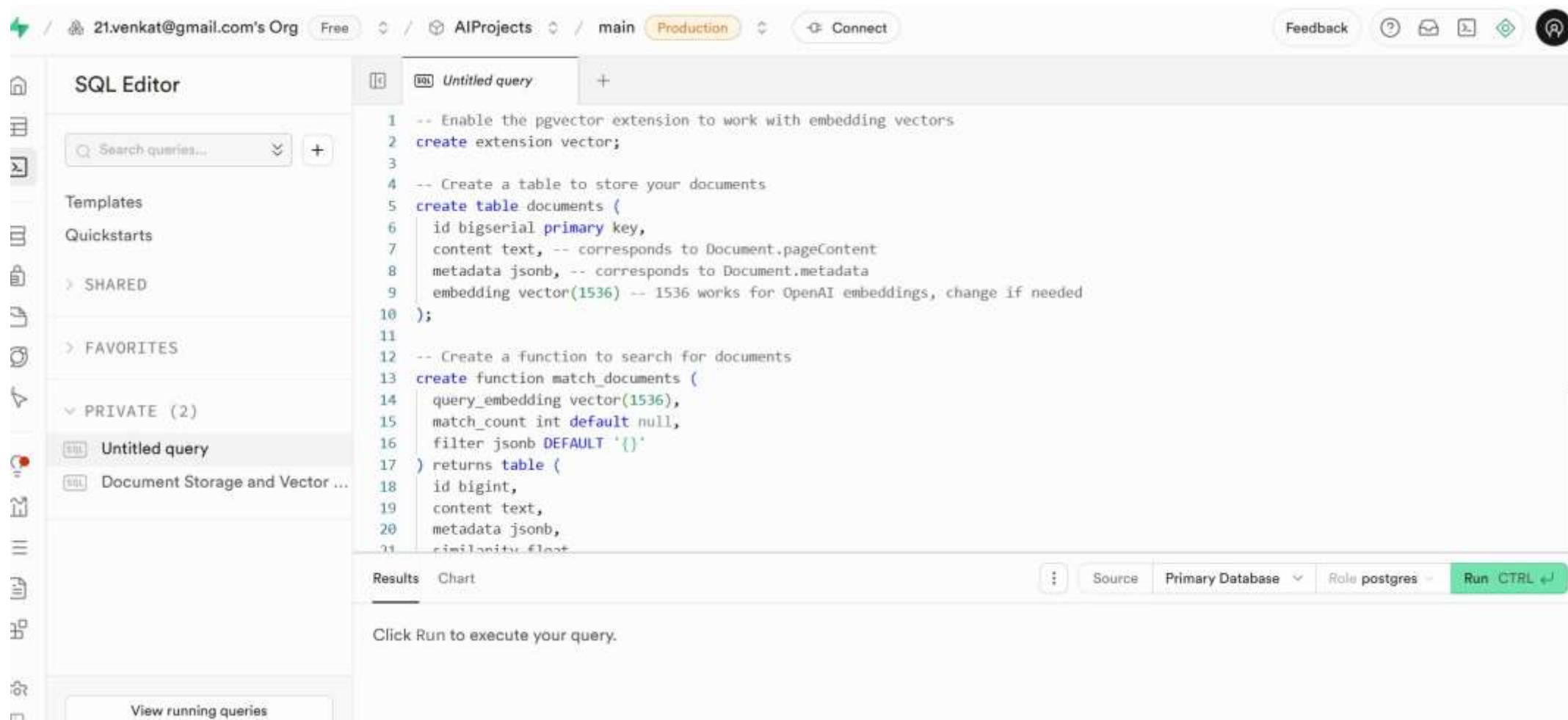
```
1  -- Enable the pgvector extension to work with embedding vectors
2  create extension vector;
3
4  -- Create a table to store your documents
5  create table documents (
6    id bigserial primary key,
7    content text, -- corresponds to Document.pageContent
8    metadata jsonb, -- corresponds to Document.metadata
9    embedding vector(1536) -- 1536 works for OpenAI embeddings, change if needed
10 );
11
12 -- Create a function to search for documents
13 create function match_documents (
14   query_embedding vector(1536),
15   match_count int default null,
16   filter jsonb DEFAULT '{}'
17 );
```

RAG Workflow

SQL Editor in Supabase



RAG Workflow



The screenshot displays a web-based SQL Editor interface. The top navigation bar includes the user email '21.venkat@gmail.com's Org', a 'Free' label, the project name 'AIProjects', the database 'main', and a 'Production' environment tag. A 'Connect' button is also present. On the left sidebar, there are sections for 'Templates', 'Quickstarts', 'SHARED', 'FAVORITES', and 'PRIVATE (2)'. The main editor area shows a query in a file named 'Untitled query'. The query is as follows:

```
1 -- Enable the pgvector extension to work with embedding vectors
2 create extension vector;
3
4 -- Create a table to store your documents
5 create table documents (
6   id bigserial primary key,
7   content text, -- corresponds to Document.pageContent
8   metadata jsonb, -- corresponds to Document.metadata
9   embedding vector(1536) -- 1536 works for OpenAI embeddings, change if needed
10 );
11
12 -- Create a function to search for documents
13 create function match_documents (
14   query_embedding vector(1536),
15   match_count int default null,
16   filter jsonb DEFAULT '{}'
17 ) returns table (
18   id bigint,
19   content text,
20   metadata jsonb,
21   similarity float
```

Below the editor, there are tabs for 'Results' and 'Chart'. The 'Results' tab is active, showing a message: 'Click Run to execute your query.' At the bottom right, there is a 'Run' button with a keyboard shortcut 'CTRL + J'.

RAG Workflow

The screenshot shows a SQL Editor interface with a query titled "Untitled query". The query is as follows:

```
1 -- Enable the pgvector extension to store embedding vectors
2 create extension vector;
3
4 -- Create a table to store your documents
5 create table documents (
6   id bigserial primary key,
7   content text, -- corresponds to Document.pageContent
8   metadata jsonb, -- corresponds to Document.metadata
9   embedding vector(1536) -- 1536 works for OpenAI embeddings, change if needed
10 );
11
12 -- Create a function to search for documents
13 create function match_documents (
14   query_embedding vector(1536),
15   match_count int default null,
16   filter jsonb DEFAULT '{}'
17 ) returns table (
18   id bigint,
19   content text,
20   metadata jsonb,
21   similarity float
```

Two callout boxes provide instructions:

- For the second time repetition, you need to delete this extension vector line** (pointing to line 2: `create extension vector;`)
- Change the name from documents to a new name everywhere** (pointing to line 5: `create table documents (`)

The interface includes a sidebar with "SQL Editor", "Templates", "Quickstarts", and "SHARED" / "FAVORITES" sections. The bottom of the editor shows a "Run" button with a keyboard shortcut "CTRL + J".

RAG Workflow

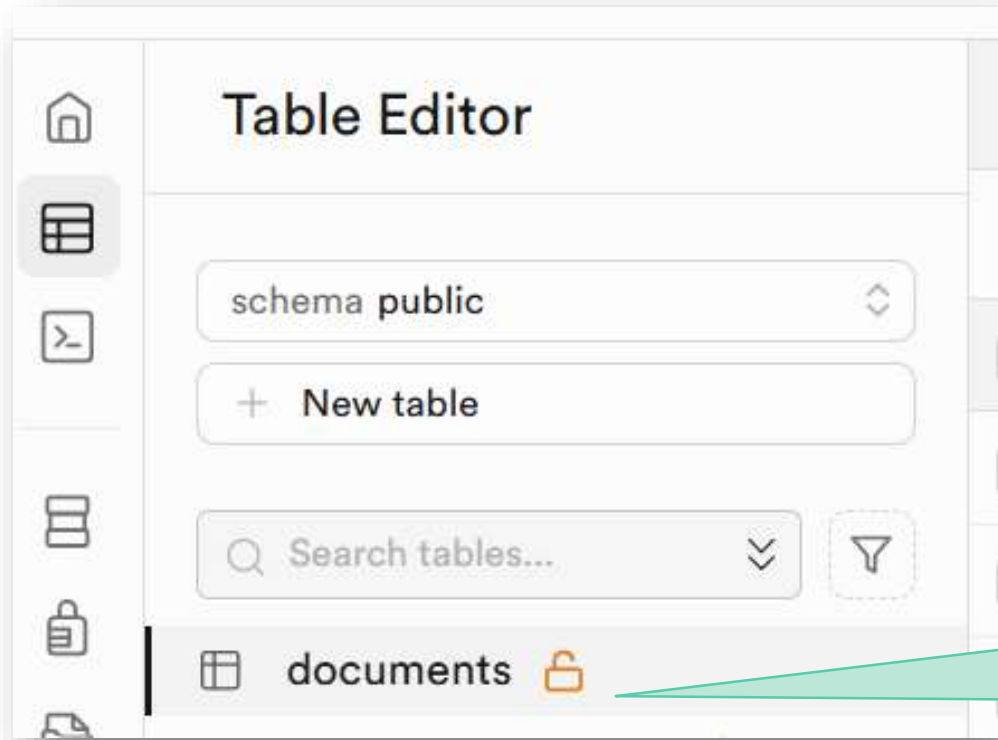


Table created for storing
the embeddings

RAG Workflow

The screenshot displays a workflow editor interface with three main sections: INPUT, a central configuration panel, and OUTPUT.

INPUT Section: Features a search bar and tabs for Schema, Table, JSON, and Binary. A Google Drive connector is selected, showing 1 item. A data preview box displays the following information:

- File Name: COI.pdf
- File Extension: pdf
- Mime Type: application/pdf
- File Size: 3.92 MB

A "Download" button is located below the data preview.

Supabase Vector Store Configuration Panel: This panel is titled "Supabase Vector Store" and includes a "Test step" button. It has tabs for Parameters, Settings, and Docs. The Parameters tab is active, showing the following settings:

- Credential to connect with:** Supabase account
- Operation Mode:** Insert Documents
- Table Name:** From list (Fixed Expression) Choose...
- Table List:** documents, n8n_chat_histories, rpc/match_documents
- No properties**
- Add Option** button

A green callout bubble points to the "documents" table in the list, containing the text "New table".

OUTPUT Section: A large empty area for the workflow's output.

Footer: Includes a "Venkata Reddy..." user indicator, a search bar, and a "I wish this node would..." feedback prompt.

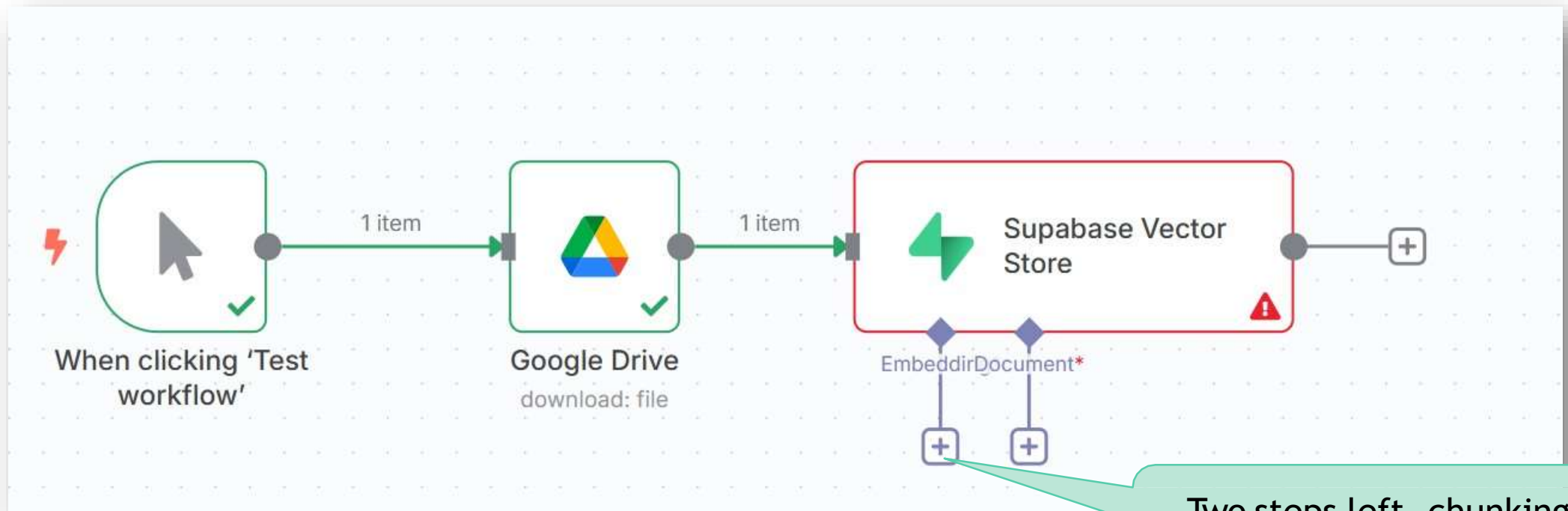
RAG Workflow

The screenshot displays a workflow editor with three main sections: INPUT, a central configuration panel, and OUTPUT.

- INPUT Section:** Features a "Google Drive" dropdown menu showing "1 item". Below it, a "data" box lists file details: File Name: COI.pdf, File Extension: pdf, Mime Type: application/pdf, and File Size: 3.92 MB. A "Download" button is at the bottom.
- Central Configuration Panel:** Titled "Supabase Vector Store" with a "Test step" button. It has tabs for "Parameters", "Settings", and "Docs".
 - Parameters Tab:**
 - "Credential to connect with": Supabase account
 - "Operation Mode": Insert Documents
 - "Table Name": From list (dropdown) and documents (dropdown)
 - "Embedding Batch Size": 200
 - Options:**
 - "Query Name": match_documents
- OUTPUT Section:** A large empty area for the final result.

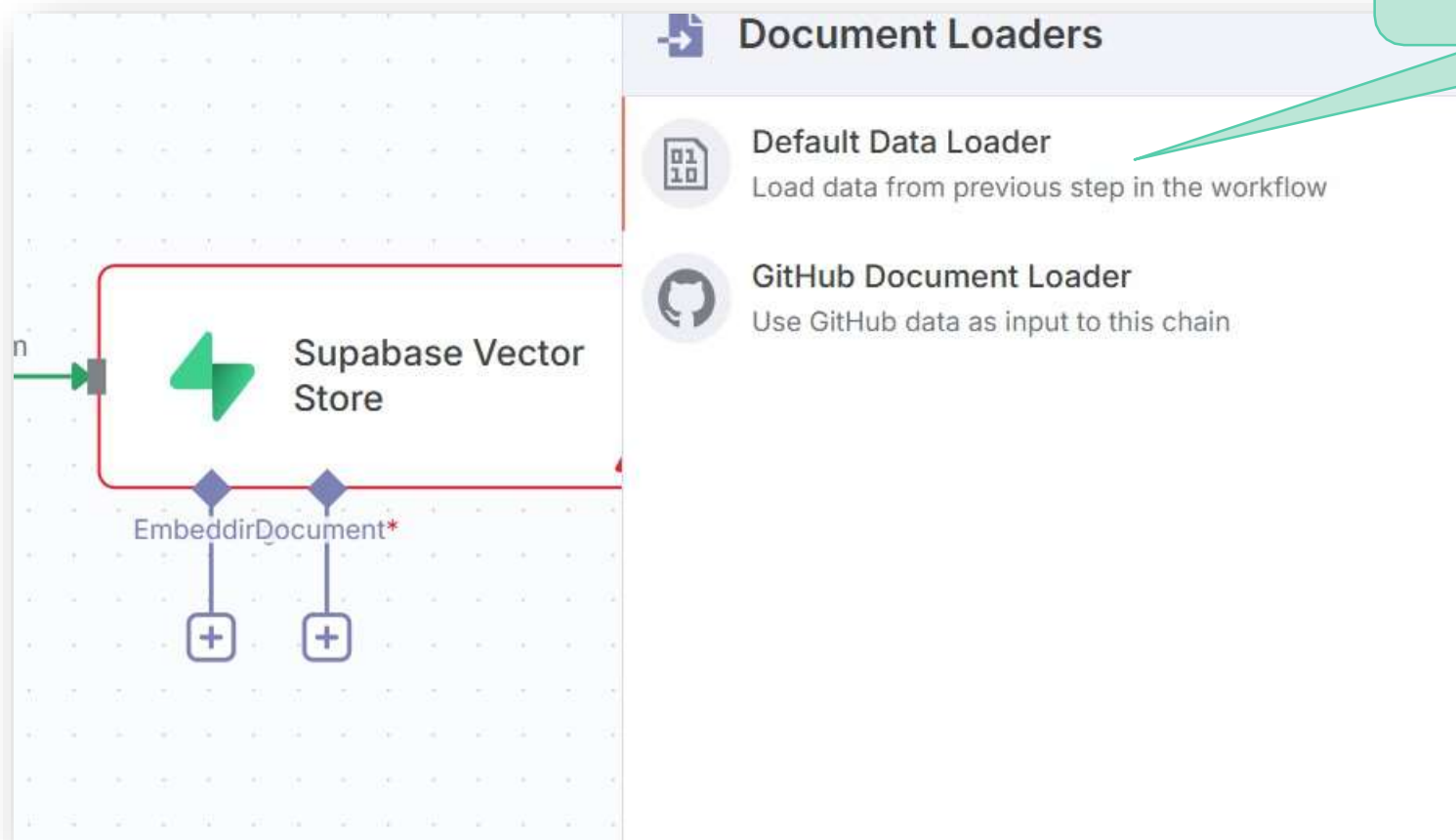
A green callout bubble points to the "Query Name" field with the text: "This should be adjusted based on the table name".

RAG Workflow



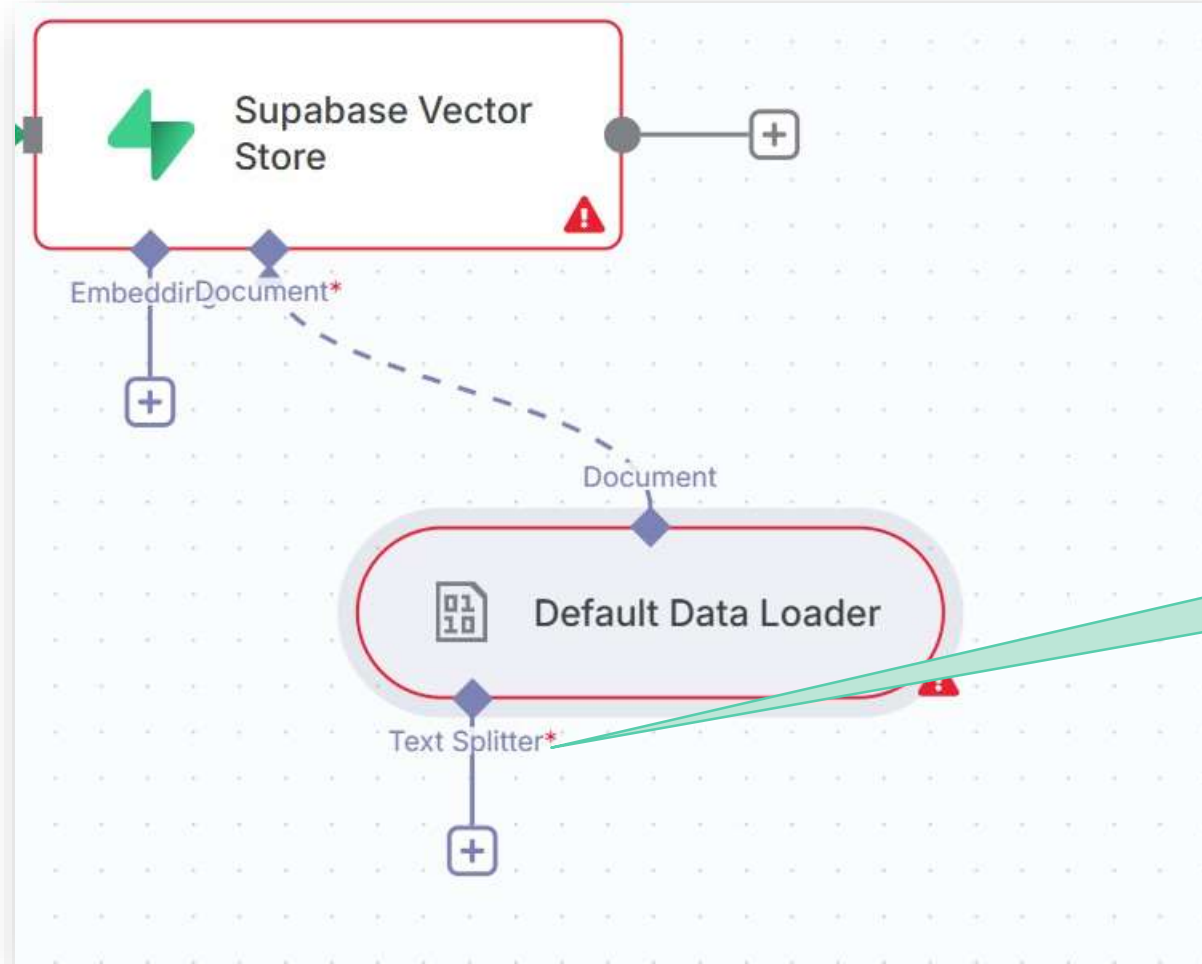
Two steps left, chunking and embedding.

RAG Workflow

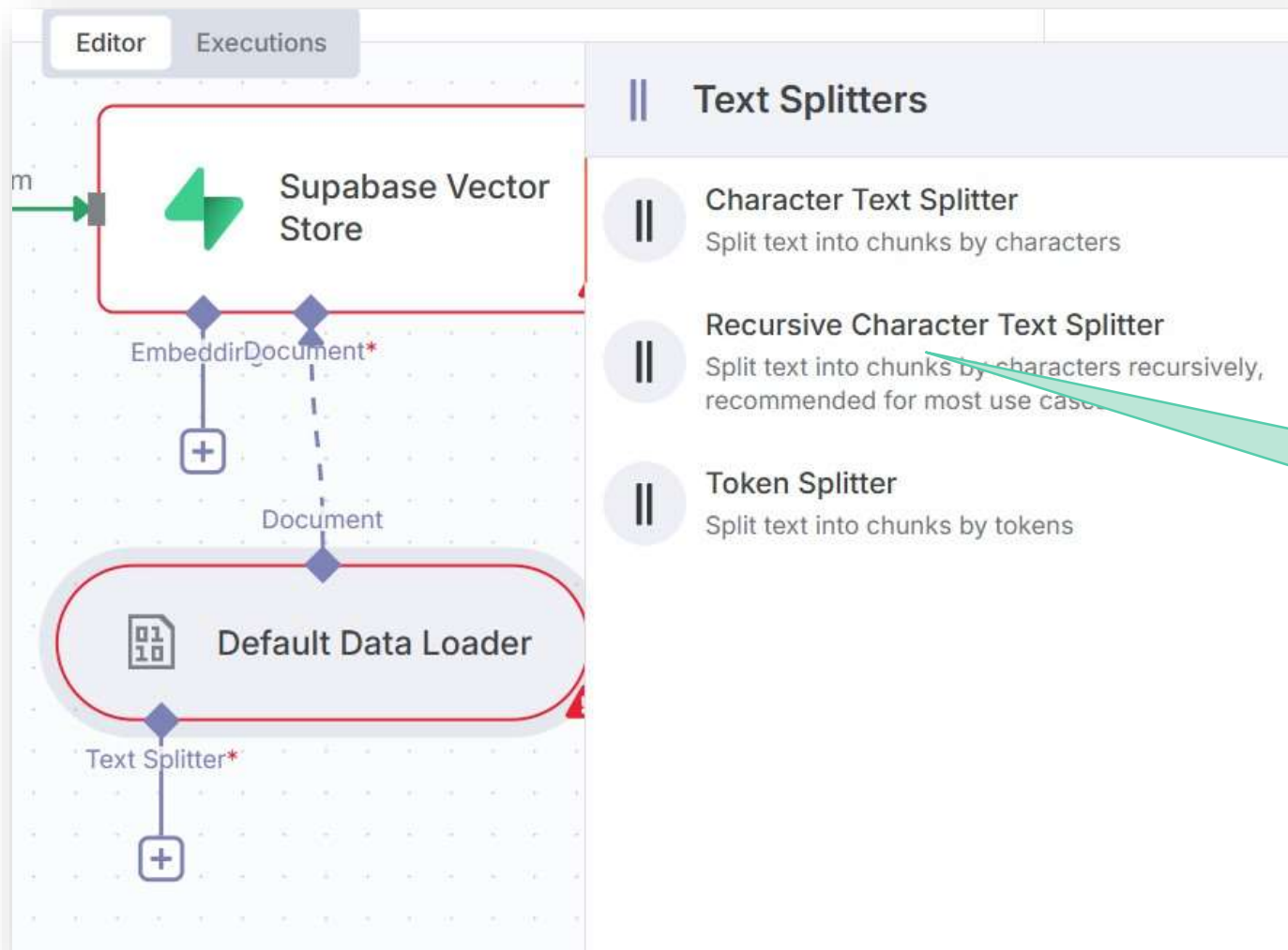


Choose Default Data Loader

RAG Workflow



RAG Workflow



RAG Workflow

The screenshot displays a workflow editor interface for a Retrieval-Augmented Generation (RAG) system. The central focus is a configuration panel for a 'Recursive Character Text splitter' node. This panel includes tabs for 'Parameters', 'Settings', and 'Docs'. Under the 'Parameters' tab, the 'Chunk Size' is set to 1000 and the 'Chunk Overlap' is set to 100. A green callout bubble points to these two settings with the text 'Chunk size and Chunk overlap'. The 'Options' section below shows 'No properties' and an 'Add Option' button. To the left of the configuration panel, the 'INPUT' section shows a 'Google Drive' data source with one item, 'COI.pdf', which is 3.92 MB in size. The 'OUTPUT' section on the right is currently empty. The interface also features a 'Back to canvas' button at the top left and a status bar at the bottom with the text 'I wish this node would...'.

Back to canvas

INPUT

Google Drive

1 item

data

File Name:
COI.pdf
File Extension:
pdf
Mime Type:
application/pdf
File Size:
3.92 MB

Download

Recursive Character Text splitter

Parameters Settings Docs

Chunk Size
1000

Chunk Overlap
100

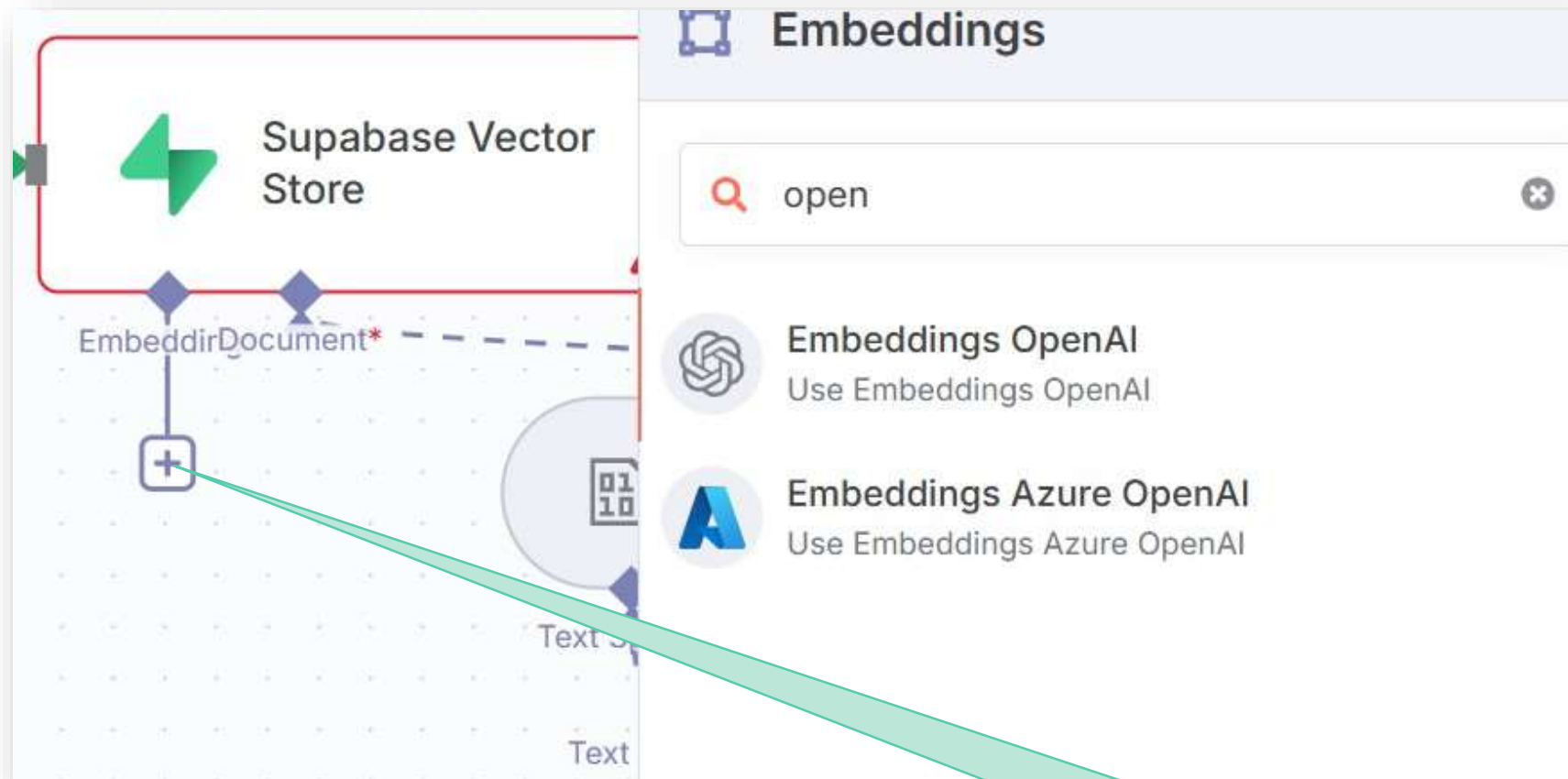
Options
No properties
Add Option

OUTPUT

Chunk size and Chunk overlap

I wish this node would...

RAG Workflow



Choose OpenAI embeddings

RAG Workflow

The screenshot displays a workflow editor interface with three main sections: INPUT, Mapping, and OUTPUT. The INPUT section shows a Google Drive connection with one item, 'data', which is a PDF file named 'COI.pdf' (3.92 MB). The Mapping section is currently active, showing the configuration for the 'Embeddings OpenAI' node. The OUTPUT section is empty. A green callout box points to the 'Model' dropdown menu, which is set to 'text-embedding-3-small'. The callout box contains the text 'Choose the model'.

INPUT | Mapping | Debugging | Schema | Table | JS

Google Drive

1 item

data

File Name:
COI.pdf
File Extension:
pdf
Mime Type:
application/pdf
File Size:
3.92 MB

Download

Embeddings OpenAI

Parameters | Settings | Docs

Credential to connect with
OpenAi account

Model
text-embedding-3-small

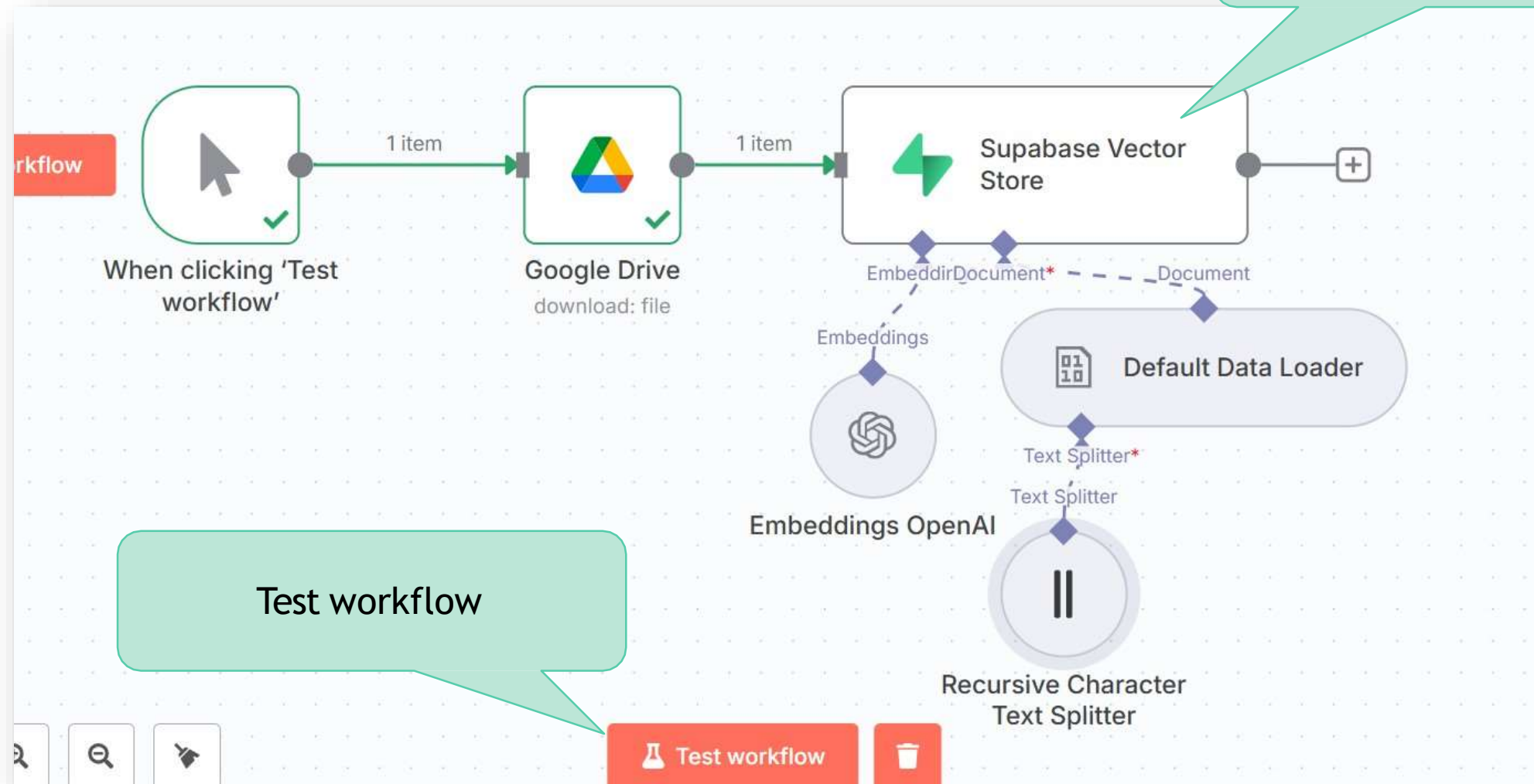
Options
No properties
Add Option

Choose the model

OUTPUT

I wish this node would...

RAG Workflow



RAG Workflow

The screenshot shows a web application interface for a 'Table Editor'. The top navigation bar includes the user '21.venkat@gmail.com's Org', a 'Free' tier indicator, and project settings for 'AIProjects' and 'main' in 'Production' mode. A 'Connect' button is also visible. The left sidebar contains icons for home, database, tables, documents, and other functions. The main area displays a table with columns: 'id' (int8), 'content' (text), 'metadata' (jsonb), and 'embedding' (vector). The table contains 12 rows of data, each representing a document snippet. A callout box points to the 'embedding' column, indicating that there are 1025 vectors in total. The bottom of the interface shows pagination controls: 'Page 1 of 11', '100 rows', and '1,025 records'.

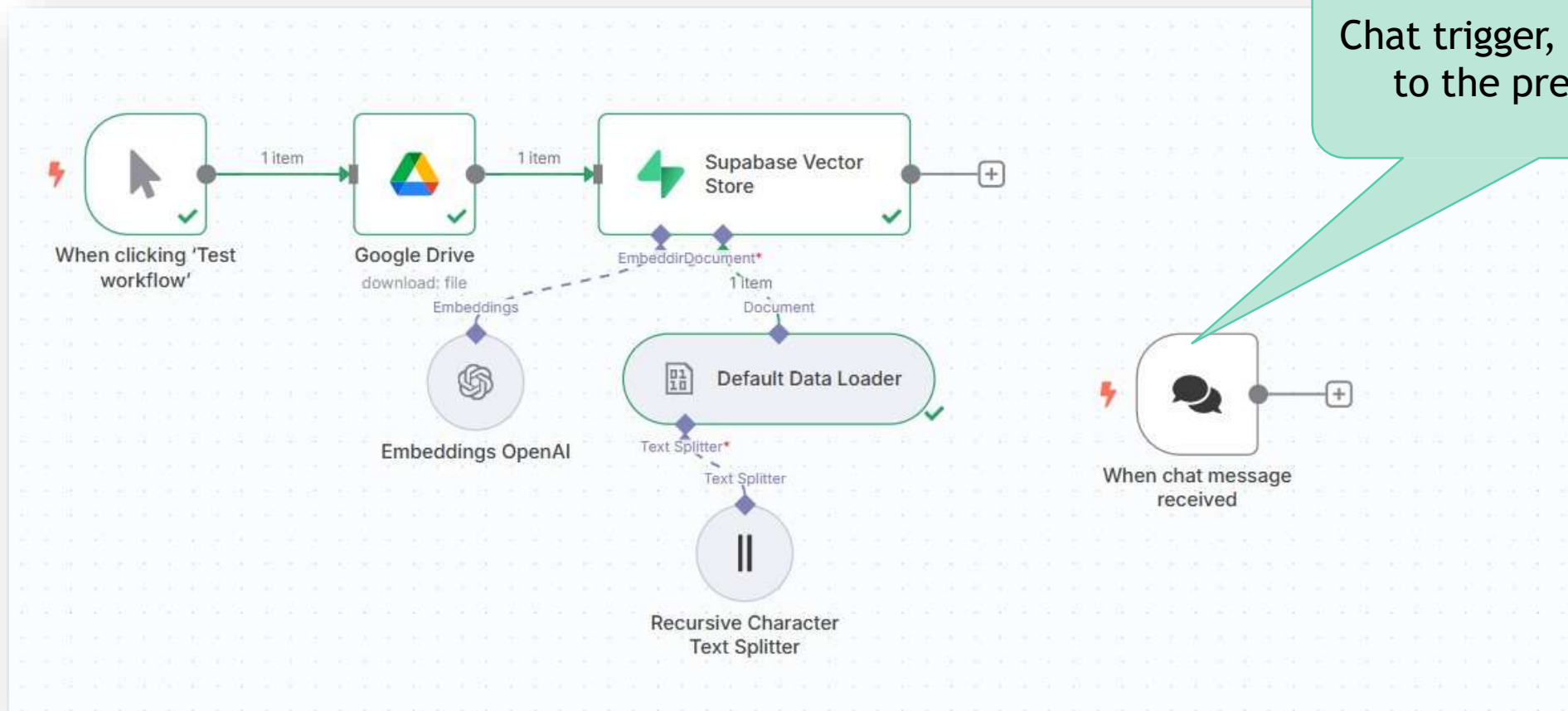
	id	content	metadata	embedding
1	1	THE CONSTITUTION OF INDIA [As on 26	{\"loc\":{\"lines\":{\"to\":31,\"from\":4}},\"pdf\":{\"in	[0.0105038965,0.027720
2	2	been provided respectively in Appendix II	{\"loc\":{\"lines\":{\"to\":59,\"from\":27}},\"pdf\":{\"i	[0.025550071,0.0247279
3	3	ourth Schedules and supplemental, incide	{\"loc\":{\"lines\":{\"to\":85,\"from\":56}},\"pdf\":{\"	[0.048630767,0.0554957
4	4	rtunity in matters of public employment.	{\"loc\":{\"lines\":{\"to\":112,\"from\":83}},\"pdf\":{\"	[0.056330446,0.026062
5	5	ltural and Educational Rights 29. Protectic	{\"loc\":{\"lines\":{\"to\":143,\"from\":108}},\"pdf\":{\"	[0.06484092,0.07169864
6	6	in any area. 35. Legislation to give effect t	{\"loc\":{\"lines\":{\"to\":169,\"from\":138}},\"pdf\":{\"	[0.033362765,0.0423204
7	7	46. Promotion of educational and econom	{\"loc\":{\"lines\":{\"to\":207,\"from\":168}},\"pdf\":{\"	[0.004819405,0.0455451
8	8	57. Eligibility for re-election. 58. Qualifica	{\"loc\":{\"lines\":{\"to\":236,\"from\":202}},\"pdf\"	[0.001150
9	9	sident. 70. Discharge of President's functi	{\"loc\":{\"lines\":{\"to\":265,\"from\":232}},\"pdf\"	[-0.01193
10	10	ach census. 83. Duration of Houses of Par	{\"loc\":{\"lines\":{\"to\":282,\"from\":263}},\"pdf\"	[0.011139719,0.018
11	11	offices of Speaker and Deputy Speaker. 9:	{\"loc\":{\"lines\":{\"to\":320,\"from\":280}},\"pdf\"	[0.0065840087,0.022381
12	12	when disqualified. Powers, Privileges and	{\"loc\":{\"lines\":{\"to\":353,\"from\":315}},\"pdf\":{\"	[0.04039925,0.06018476

RAG Workflow

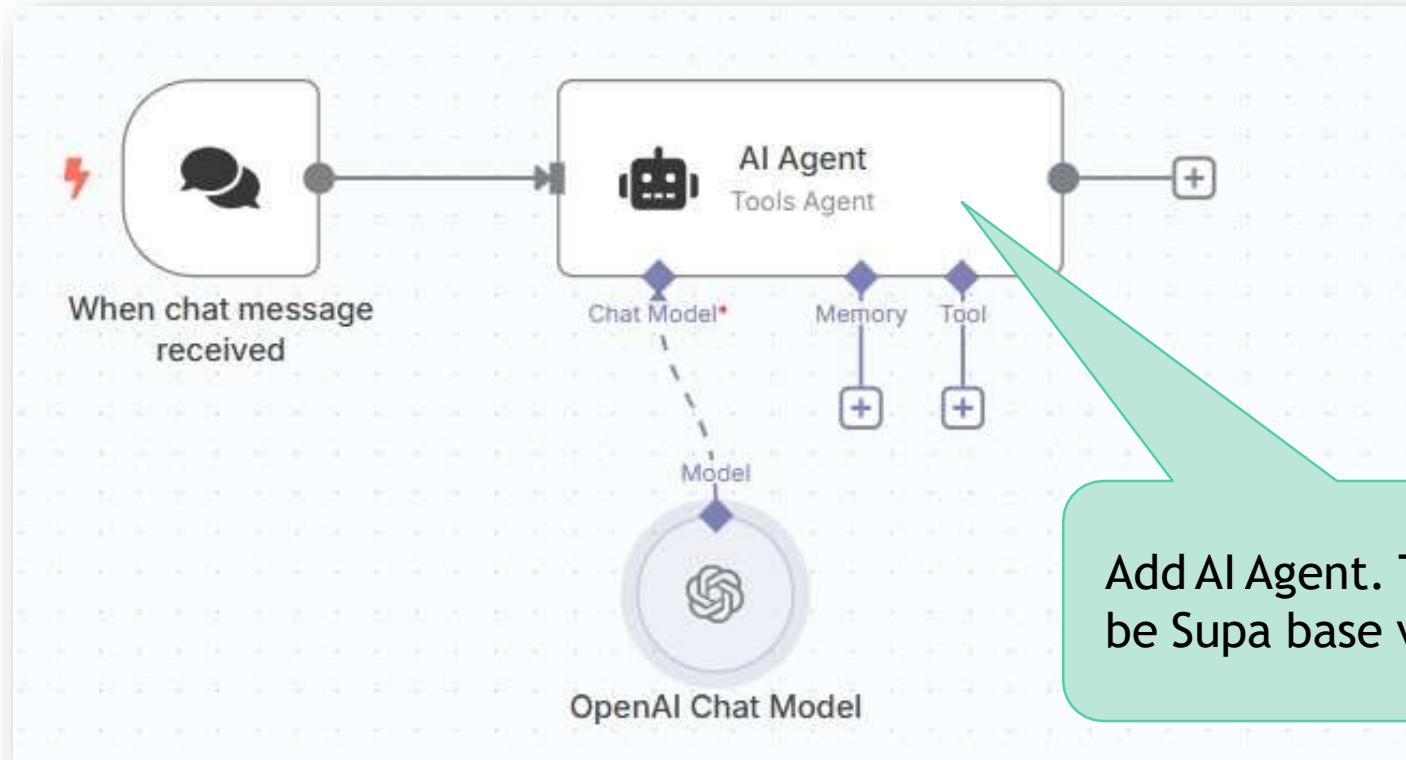


RAG Workflow

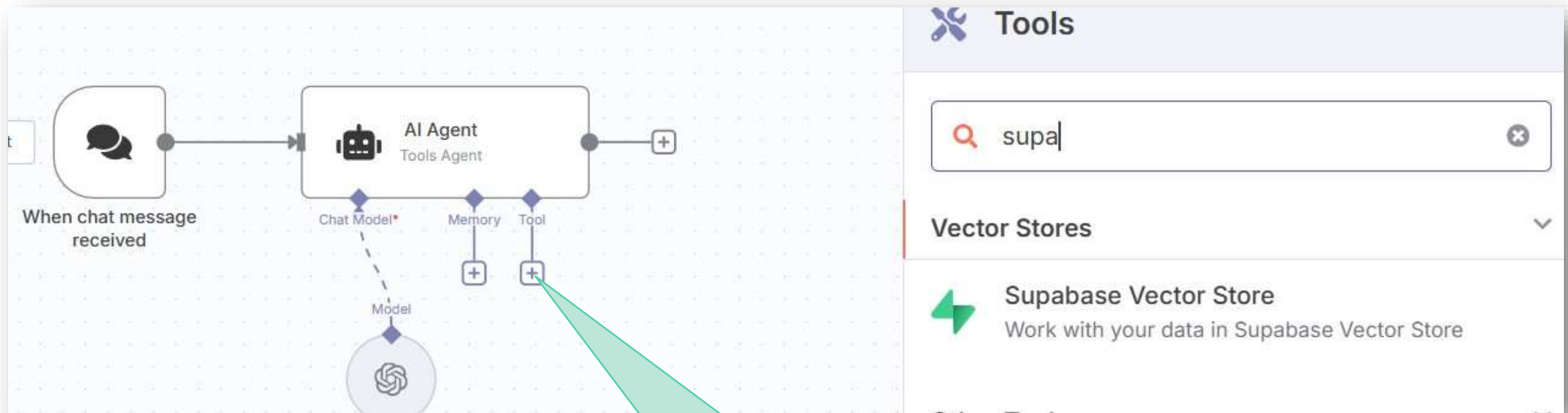
Chat trigger, not connected to the previous nodes



RAG Workflow



RAG Workflow



The tool will be Supabase vector store.

RAG Workflow

The screenshot shows the 'Supabase Vector Store1' configuration panel in a workflow editor. The panel is divided into 'Parameters' and 'Settings' tabs. The 'Parameters' tab is active, showing fields for 'Credential to connect with' (Supabase account), 'Operation Mode' (Retrieve Documents (As Tool for AI Agent)), 'Name' (COL_Database), 'Description' (Answer the questions asked in the chat by using vector store data), 'Table Name' (From list: documents), and 'Limit' (4). A callout bubble points to the 'Name' and 'Description' fields with the text: 'Give a name and write the description, choose the table name'. The 'INPUT' section on the left shows 'No input data yet' and an 'Execute previous nodes' button. The 'OUTPUT' section on the right shows 'Execute this node to view data or set mock data'.

Back to canvas +

INPUT Mapping Debugging

No input data yet

Execute previous nodes

(From the earliest node that needs it ?)

Supabase Vector Store1

Parameters Settings Docs

Credential to connect with
Supabase account

Operation Mode
Retrieve Documents (As Tool for AI Agent)

Name
COL_Database

Description
Answer the questions asked in the chat by using vector store data

Table Name
From list documents

Limit
4

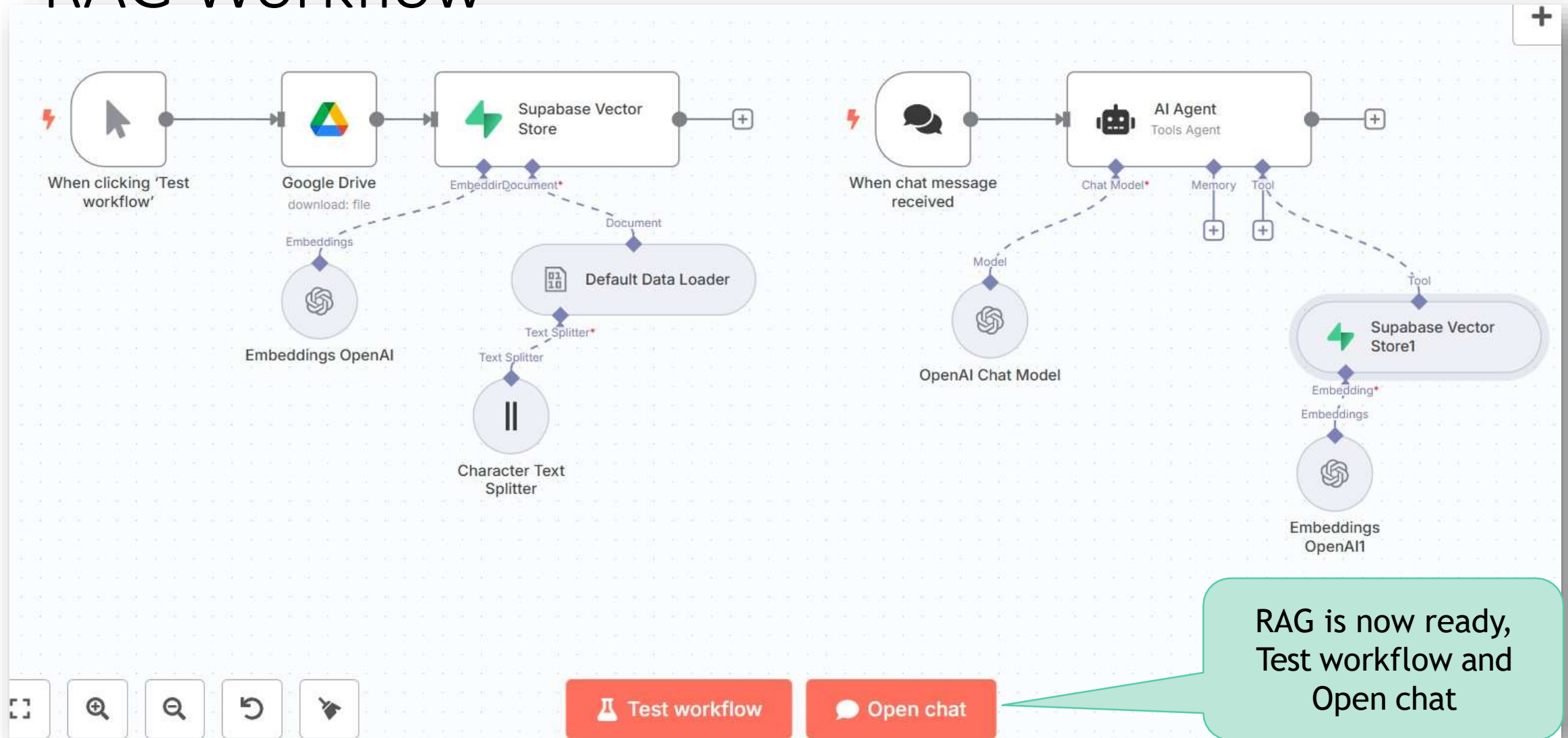
Embedding *

OUTPUT

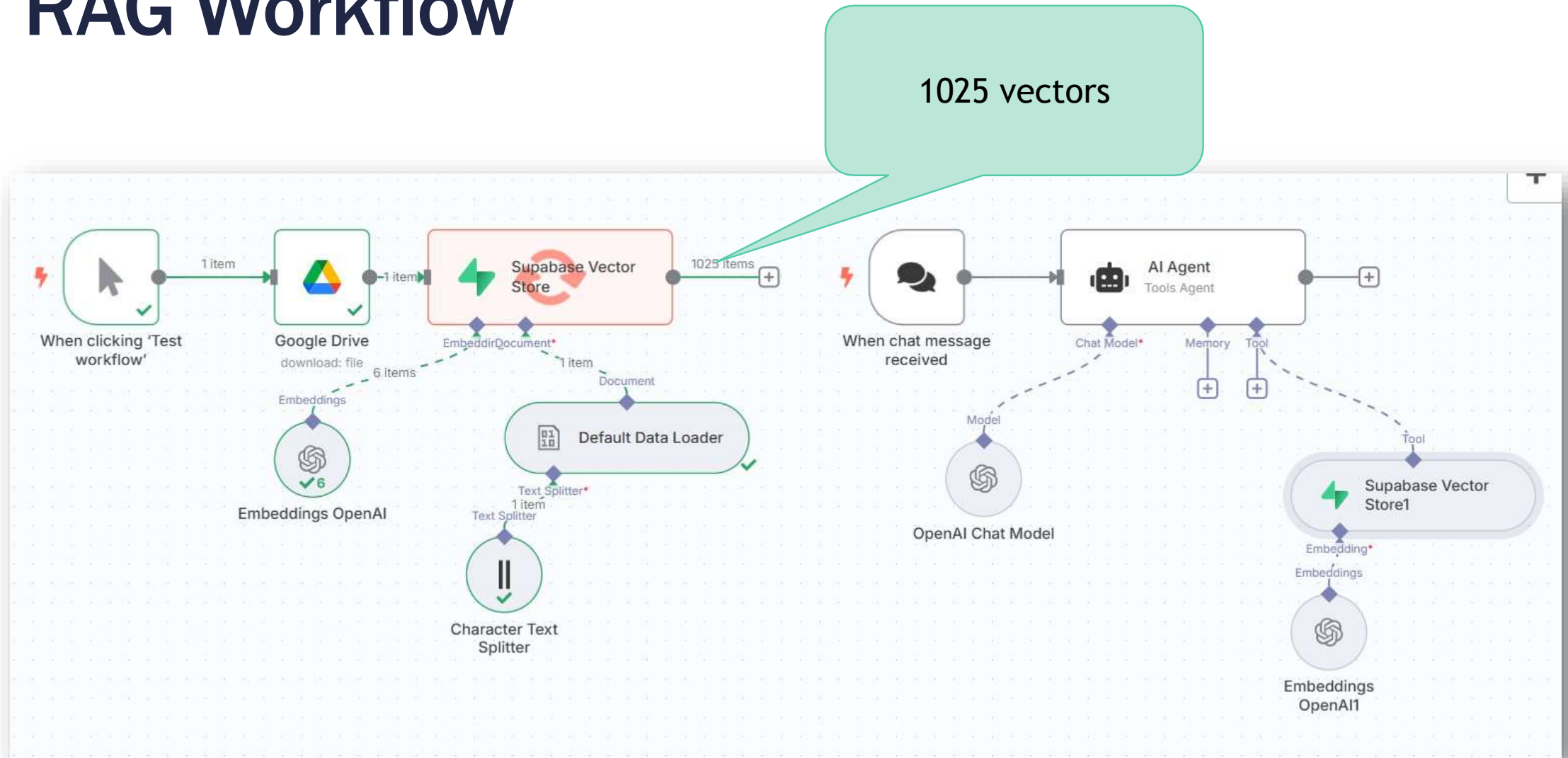
Execute this node to view data or set mock data

Give a name and write the description, choose the table name

RAG Workflow



RAG Workflow



RAG Workflow

The screenshot displays a RAG workflow interface with two main panels. The left panel, titled 'Chat', shows a session ID '44518cb33fd8426b86d85cae732...' and a user query 'What are the main rules in COI?'. The response from the AI agent is: 'The main rules in Cooperative Societies (COI) include: 1. **Election Practices:** Any corrupt practices during the election of members of the board or office bearers are prohibited. 2. **Co-Opted Members:** The number of co-opted members on the board cannot...'. The right panel, titled 'Latest Logs from AI Agent node', shows a log entry for the 'OpenAI Chat Model' with a response time of 868ms, a start time of 8:46:27 PM, and 81 Tokens. The log entry is expanded to show the 'Input' and 'Output' sections. The 'Input' section contains the text 'Human: What are the main rules in COI?'. The 'Output' section is currently empty.

Chat

Session 44518cb33fd8426b86d85cae732...

What are the main rules in COI?

The main rules in Cooperative Societies (COI) include:

- Election Practices:** Any corrupt practices during the election of members of the board or office bearers are prohibited.
- Co-Opted Members:** The number of co-opted members on the board cannot...

Type a message, or press 'up' arrow for previous one

Latest Logs from AI Agent node

OpenAI Chat Model
868ms | Started at 8:46:27 PM | 81 Tokens

Input

Human: What are the main rules in COI?

Output

Chat results

RAG Workflow

What are the fundamental rights ?

Fundamental rights are a set of rights that are recognized as being essential for the development and well-being of individuals in a society. In the context of the Constitution of India, the fundamental rights are enumerated in Part III and include the following categories:

1. Right to Equality:

- Equality before law.

^ Type a message, or press 'up' arrow for previous one



Chat results

RAG Workflow

Click on Agent>>Check Logs to see what's happening behind the scenes.

The screenshot displays the AI Agent configuration interface. On the left, the 'INPUT' section shows a node 'When chat message received' with a single item. The item details are: sessionId: 44518cb33fd8426b86d85cae732c5498, action: sendMessage, and chatInput: What are the fundamental rights?. Below this, the 'Variables and context' section is visible.

In the center, the 'AI Agent' configuration panel is open, showing the 'Parameters' tab. It includes a 'Test step' button, a tip about getting a feel for agents, and a 'Source for Prompt (User Message)' dropdown set to 'Connected Chat Trigger Node'. The 'Prompt (User Message)' field contains a JSON template: `{{ $json.chatInput }}`. Below the prompt, there is a toggle for 'Require Specific Output Format' which is currently off, and an 'Options' section with 'No properties' and an 'Add Option' button.

On the right, the 'Output' tab is selected, showing the 'Logs' section. The log entry shows the output: 'Fundamental rights are a set of rights that are recognized as being essential for the development and well-being of individuals in a society. In the context of the Constitution of India, the fundamental rights are enumerated in Part III and include the following categories: 1. **Right to Equality:** - Equality before law. - Prohibition of discrimination on grounds of religion, race, caste, sex, or place of birth. - Equality of opportunity in matters of public employment. - Abolition of untouchability. - Abolition of titles. 2. **Right to Freedom:** - Protection of certain r...'. A green checkmark icon is visible next to the 'Logs' tab.

RAG Workflow

The image displays a RAG (Retrieval-Augmented Generation) workflow interface. On the left, the 'AI Agent' configuration panel is visible, featuring tabs for 'Parameters', 'Settings', and 'Docs'. The 'Parameters' tab is active, showing a 'Test step' button and a tip about getting a feel for agents. The 'Source for Prompt (User Message)' is set to 'Connected Chat Trigger Node'. The 'Prompt (User Message)' is defined as `fx [{{ $json.chatInput }}]`. The user message is 'What are the fundamental rights?'. The 'Require Specific Output Format' toggle is turned off. The 'Options' section shows 'No properties' and an 'Add Option' button.

The main workspace on the right shows the workflow execution. The 'AI Agent' node is expanded, revealing a sequence of steps: 'OpenAI Chat Model', 'Supabase Vector Store1', 'Embeddings OpenAI1', and 'OpenAI Chat Model'. The 'Supabase Vector Store1' node is highlighted with a green arrow pointing to a 'Logs' box. The 'Logs' box displays the input and output of the workflow. The input is a JSON object: `{ "query": "What are the fundamental rights?" }`. The output is a JSON object: `{ "response": [ { "type": "text", "text": "\\\"pageContent\\\":\\\"ourth Schedules and supplemental, incidental \\\"and consequential matters \\\"PART II \\\"STIPENDS\\\" \\\"C. Signature\\\""} ] }`.

Logs

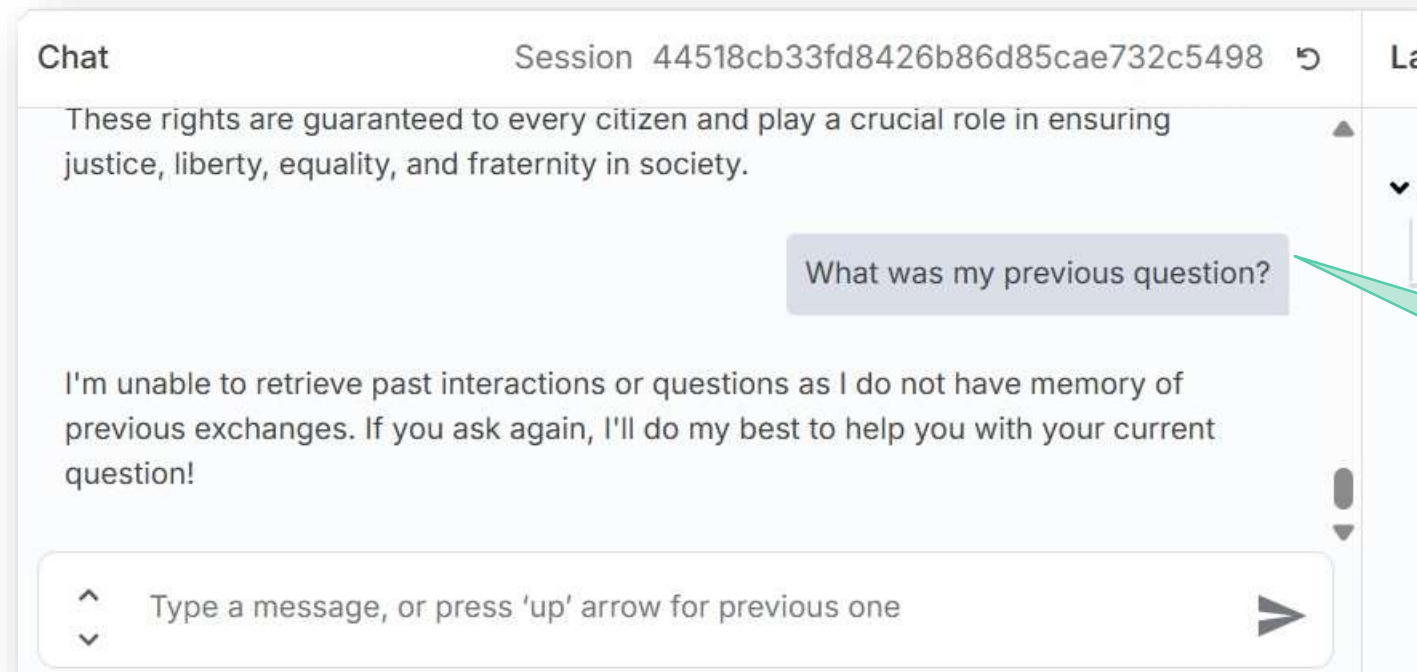
RAG Workflow

The image displays a RAG workflow configuration interface. On the left, the 'AI Agent' configuration panel is visible, featuring tabs for 'Parameters', 'Settings', and 'Docs'. The 'Parameters' tab is active, showing a 'Source for Prompt (User Message)' dropdown set to 'Connected Chat Trigger Node', a 'Prompt (User Message)' text area containing the JSON path `{{ $.chatInput }}`, and a 'Require Specific Output Format' toggle switch. Below these are 'Options' and a 'No properties' message with an 'Add Option' button. At the bottom, there are buttons for 'Chat Model *', 'Memory', and 'Tool'.

On the right, the 'Output' and 'Logs' tabs are shown, with 'Logs' selected. A green arrow points from the 'Supabase Vector Store1' node in the workflow diagram to the 'Logs' tab. The 'Logs' tab displays a JSON response:

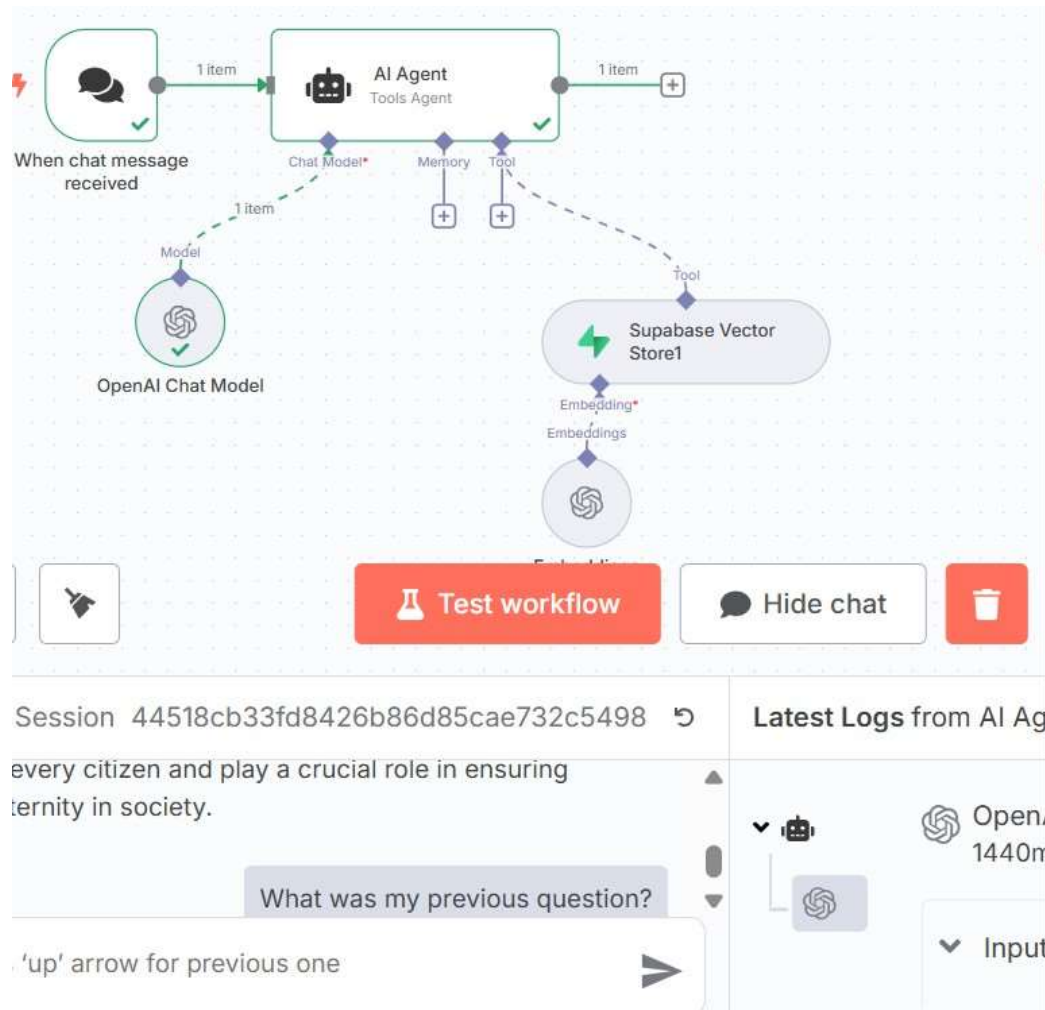
```
{
  "response": [
    -0.001417977618984878,
    0.04703642055392265,
    0.06474065035581589,
    0.06871863454580307,
    -0.01436009630560875,
    -0.03197689726948738,
    -0.023846065625548363,
    0.016163304448127747,
    -0.010442215949296951,
    -0.031802039593458176,
    0.03682916611433029,
    -0.047429848462343216,
    -0.05114554986357689,
    0.013988526538014412,
    0.009464112110435963,
    -0.012141603976488113,
    -0.00164474465418607,
    0.03293860703706741,
    0.05704696103930473,
  ]
}
```

RAG Workflow



This RAG lags memory, lets add memory using another SQL server

RAG Workflow



Memory

Memory allows an AI model to remember and reference past interactions with it

For beginners

- Simple Memory**
Stores in n8n memory, so no credentials required

Other memories

- MongoDB Chat Memory**
Stores the chat history in MongoDB collection.
- Motorhead**
Use Motorhead Memory
- Postgres Chat Memory**
Stores the chat history in Postgres table.
- Redis Chat Memory**
Stores the chat history in Redis.
- Xata**
Use Xata Memory

RAG Workflow

← Back to canvas

Postgres Chat Memory

Parameters Settings Docs

When received

A s

A a

A c

Variables

44518cb33fd8426b86d85cae732c5498

Credential to connect with

Postgres account

Postgres account

Postgres

+ Create new credential

`fx {{ $json.sessionId }}`

Table Name

n8n_chat_histories

Context Window Length

5

How many past interactions the model receives as context

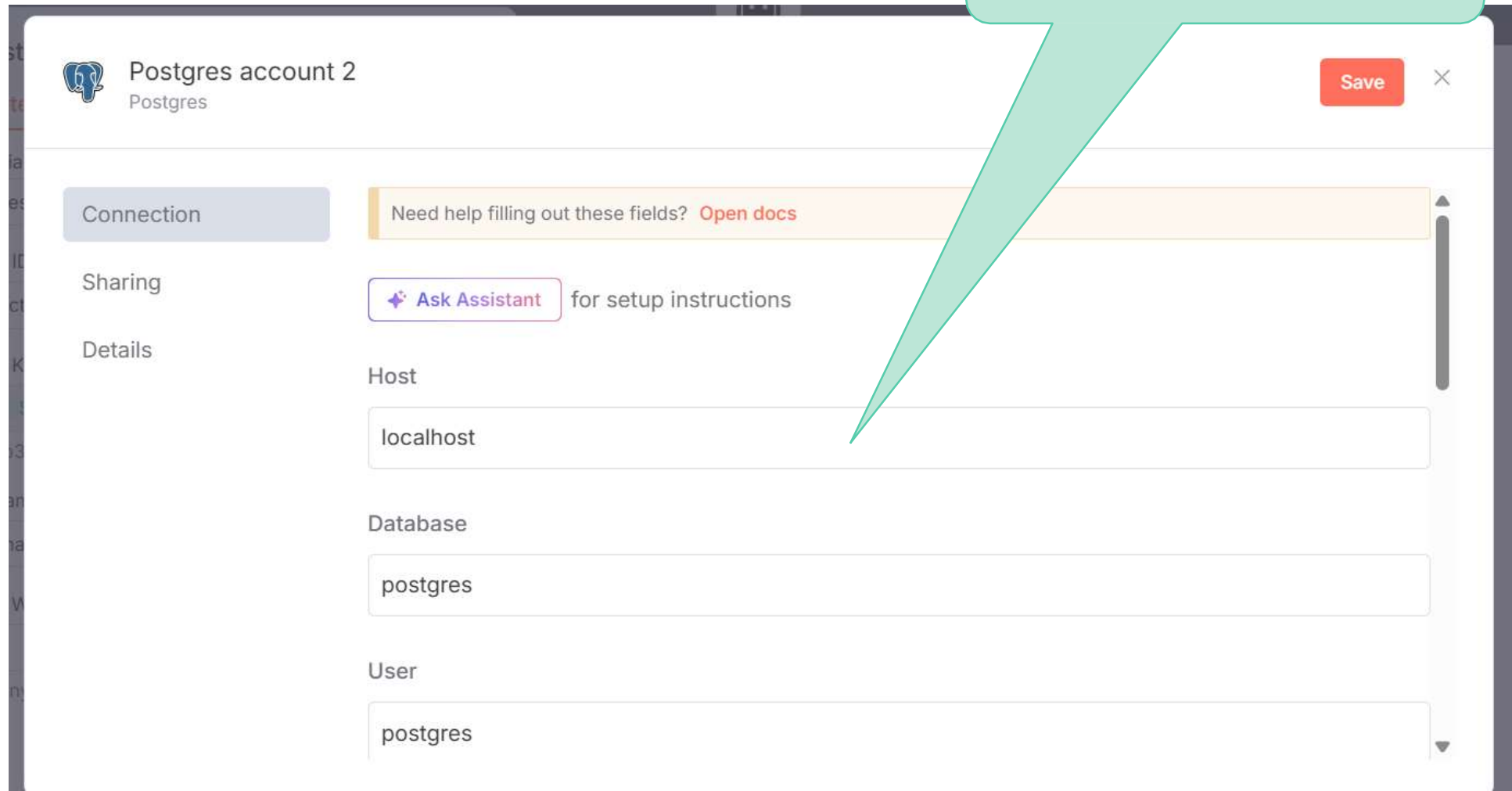
For Postgres credentials

OUTPUT

once

un

RAG Workflow



The screenshot shows the 'Postgres account 2' setup page in Supabase. The interface includes a sidebar with 'Connection', 'Sharing', and 'Details' tabs. The 'Connection' tab is active, showing fields for Host, Database, and User. A callout box points to the 'localhost' value in the Host field.

Postgres account 2
Postgres

Save

Connection

Need help filling out these fields? [Open docs](#)

Sharing

Details

[Ask Assistant](#) for setup instructions

Host

localhost

Database

postgres

User

postgres

You will get these details from Supabase

RAG Workflow

Connect on Supabase

The screenshot shows the Supabase AIProjects dashboard. The top navigation bar includes the user's email (21.venkat@gmail.com's Org), a 'Free' label, the project name 'AIProjects', a 'main' branch selector, a 'Production' environment selector, and a 'Connect' button. A green callout bubble points to the 'Connect' button with the text 'Connect on Supabase'. Below the navigation bar, the main content area displays 'AIProjects' with a 'NANO' label. To the right, there are statistics for 'Tables' (2), 'Functions' (0), and 'Replicas' (0), along with a 'Project Status' indicator showing a green dot. A welcome message states: 'Welcome to your new project. Your project has been deployed on its own instance, with its own API all set up and ready to use.' At the bottom right, a table shows a single task:

	id	task	status
☐	1	Create a project	Complete

RAG Workflow

Copy these details to
postgres

Transaction pooler Shared Pooler

Ideal for stateless applications like serverless functions where interaction with Postgres is brief and isolated.

```
postgresql://postgres.jmihdvwktcxhrmzeluib:[YOUR PASSWORD]@
```

Does not support PREPARE statements

View parameters

```
host: aws-0-ap-south-1.pooler.supabase.com
```

```
port: 6543
```

```
database: postgres
```

```
user: postgres.jmihdvwktcxhrmzeluib
```

```
pool_mode: transaction
```

For security reasons, your database password is never shown.



Suitable for a large number of connected clients

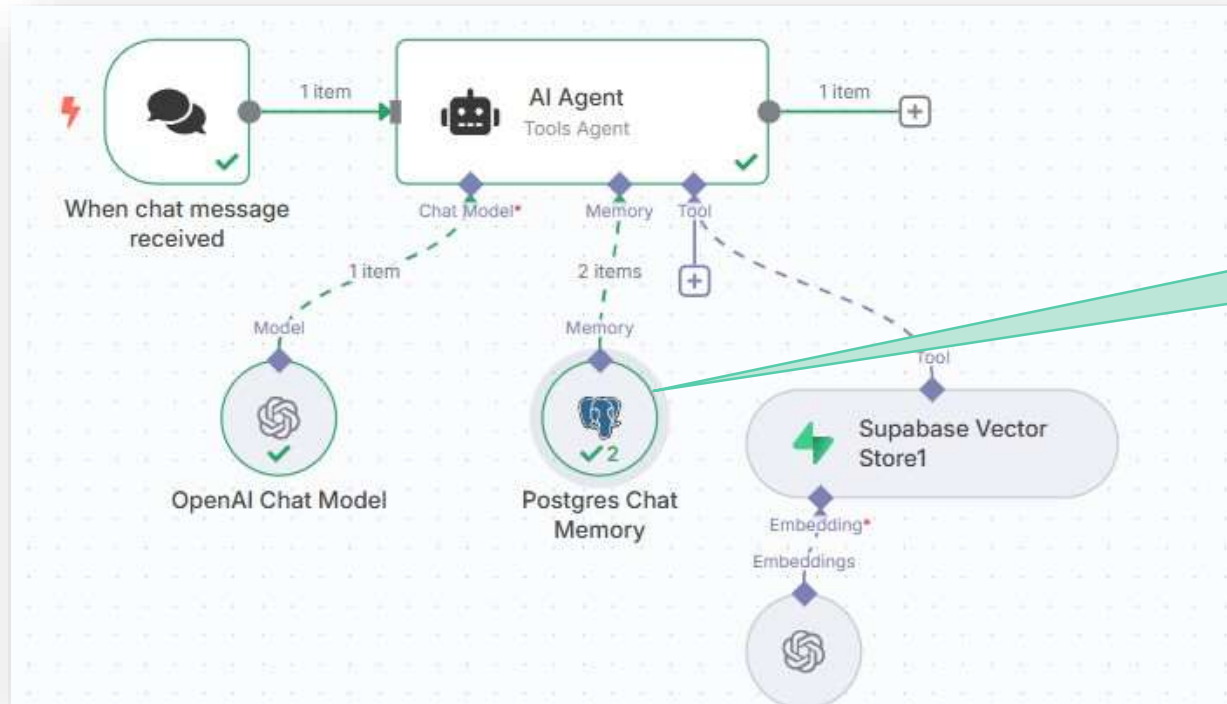
Pre-warmed connection pool to Postgres



IPv4 compatible

Transaction pooler connections are IPv4 proxied for free.

RAG Workflow



Memory added

Results from the memory

What was my previous question ?

Your previous question was, "What are the fundamental rights?"